



VectorSec



Euler

Penetration Testing report

April 20th, 2026

Contents

Executive summary	3
Strategic recommendations	6
Penetration testing.....	6
Supply-chain and third-party dependencies	6
Secure SDLC	6
Defense in depth	7
Vulnerability details.....	8
High severity findings	9
1.1 Geo Location blocking bypass via Spoofed x-country-code Header	9
1.2 Incomplete and Fail-Open OFAC Address Screening Coverage	10
1.3 Exposed RPC API Keys in Frontend JavaScript	11
1.4 Unauthenticated <i>eth_sendRawTransaction</i> Relay via JSON-RPC Proxy	12
1.5 Stored XSS Chain via Compromised Labels Repository (6 sinks)	14
Medium severity findings.....	18
1.6 API Rate-Limit Bypass via X-Forwarded-For Manipulation	18
1.7 Unauthenticated Access to /api/tenderly/simulate	19
Low severity findings.....	21
1.8 Client-Side OFAC Screening Cache Race Condition.....	21

Executive summary

Overall Risk Posture

The application's risk posture is assessed as **adequate**. No vulnerability was identified that allows an unauthenticated remote attacker to directly compromise user funds, take over accounts, or read or modify arbitrary user data on the production deployment as it stands today. The existing infrastructure layer, in particular Cloudflare in front of the origin and the server-side enforcement of address screening, absorbs or mitigates several of the weaker application-layer controls that were identified during testing.

Project objectives and scope

This engagement was a penetration test of the Euler's lite web application, commit hash: 3b72652fad0c3ba2a20dcdd97276539bedc9037f. All tested API endpoints:

- /api/rpc/[chainId]
- /api/screen-address
- /api/tenderly/simulate
- /api/tenderly/status
- /api/pyth/updates
- /api/labels/[file]
- /api/oracle-adapter
- /api/euler-chains
- /api/token-list
- /api/intrinsic-apy/[provider]
- /api/tos

The objectives were to identify vulnerabilities that could be exploited to compromise user funds, bypass sanctions and compliance controls, abuse paid upstream services at the organization's expense, or degrade the integrity of the application's security boundary.

The scope covered the public-facing web application and its backend API endpoints. Smart contracts, on-chain components, and underlying infrastructure (Cloudflare configuration, RPC provider accounts, CI/CD pipelines) were not in scope, though findings were surfaced where weaknesses in the application create exposure in adjacent systems. No active users or production data were targeted. Testing was conducted against staging and local environments with production-equivalent configuration where feasible.

Performed methodology

Testing combined external (black-box) and white-box approaches.

The external phase was conducted from an unauthenticated, remote attacker position with no prior knowledge of the application beyond what is publicly observable. It focused on endpoint discovery, authentication and authorization boundary testing, input handling, CORS and origin enforcement,

rate-limit and geo-location header handling, and abuse of proxied upstream services. This phase established which issues are practically reachable by an unauthenticated attacker on the open internet.

The white-box phase was conducted with full access to the application source code, including frontend (Vue/Nuxt), backend API handlers, and build configuration. It focused on tracing user-controlled input from request ingress through to rendering sinks, reviewing the trust model applied to proxied external data, auditing the RPC proxy's method allow-list and origin-handling logic, and reviewing the client-side OFAC screening composable for reactive-timing issues. Source review was used both to confirm findings surfaced during external testing and to identify issues that would be difficult or uneconomical to discover through black-box probing alone.

Severity Rating

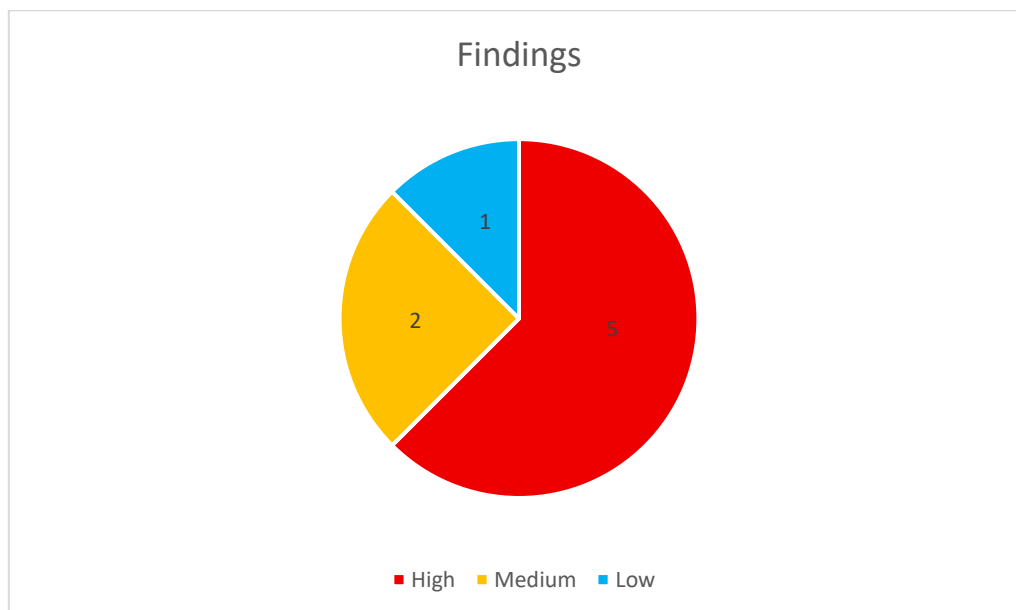
Critical – Direct, practical path to significant user-fund loss, full account takeover, or organization-wide compromise. Typically, exploitable remotely with low effort and affects many users.

High – Material impact on the organization (regulatory, financial, or reputational), unauthenticated exploitation, or clear bypass of a security control. Does not always require user interaction.

Medium – Meaningful weakening of a control, but either mitigated in the production environment, gated by infrastructure, or limited in impact to indirect cost.

Low – Narrow or edge-case issues that are theoretically exploitable but depend on precise timing, unlikely sequencing, or minor impact relative to compensating controls.

Control effectiveness summary



In aggregate, the controls that sit at the infrastructure edge and on the server-side enforcement path are doing most of the work, while application-layer controls rely on several assumptions (trusted client headers, trusted upstream data, trusted caller origin) that do not hold under adversarial conditions. Tightening these assumptions, validating or stripping client-supplied headers at ingress, applying fail-closed semantics on screening, adding schema validation on externally-sourced JSON,

and enforcing origin and authentication at the handler layer would move the posture from **adequate** to **strong** without requiring structural changes to the application.

Remediation Status Overview

Title	Severity	Status
Geo Location blocking bypass via Spoofed x-country-code Header	High	Remediated
Incomplete and Fail-Open OFAC Address Screening Coverage	High	Remediated
Exposed RPC API Keys in Frontend JavaScript	High	Remediated
Unauthenticated <i>eth_sendRawTransaction</i> Relay via JSON-RPC Proxy	High	Remediated
Stored XSS Chain via Compromised Labels Repository (6 sinks)	High	Partially remediated
API Rate-Limit Bypass via X-Forwarded-For Manipulation	Medium	Remediated
Unauthenticated Access to <code>/api/tenderly/simulate</code>	Medium	Partially remediated
Collection Client-Side OFAC Screening Cache Race Condition	Low	Remediated

Strategic recommendations

The findings in this report address specific weaknesses identified during the engagement. The recommendations in this section are programmatic rather than per-finding. They are intended to complement the remediation work already undertaken and to support a move from the current **adequate** posture towards a **strong** one over subsequent engagement cycles.

Penetration testing

- Application penetration testing should be conducted on at least an annual cadence, with an additional targeted engagement following any material architectural change, such as a new chain integration, a change to the authentication model, the introduction of a new backend service, or a significant rewrite of a trust-sensitive component.
- Infrastructure penetration testing should also be conducted annually and should be scoped separately from application testing. The current engagement explicitly excluded Cloudflare configuration, RPC provider accounts, CI/CD pipelines, and origin network exposure.
- Between annual engagements, automated vulnerability scans should be run at least quarterly against both application and infrastructure scopes. Scanning does not replace manual testing, it will not find the classes of issue that require chaining of vulnerabilities or any business logic issues. Scan output should feed into the same remediation tracking process as manual findings.

Supply-chain and third-party dependencies

- Third-party services and dependencies are now among the most common initial access vectors observed in security incidents. Every third-party service and every externally-sourced data repository the application trusts should be inventoried, and each should have a documented failure mode - what the application does if the service is unreachable, returns malformed data, or is compromised.
- Dependency scanning should be added to the CI pipeline so that vulnerable or abandoned packages are flagged before they reach production, and a software bill of materials (SBOM) should be generated per build to support incident response when a downstream dependency is disclosed as compromised.

Secure SDLC

- Running SAST at pre-commit or as a mandatory CI stage on the frontend codebase, with a rule set tuned for Vue/Nuxt rendering sinks and for web-platform URL handling, is the highest-leverage single change that can be made to the development pipeline.
- Secret scanning should be enabled in CI and, where available, at the pre-commit hook.
- Beyond automation, changes to security-sensitive code paths, authentication, ingress header handling, the RPC proxy's method allow-list, rendering of data sourced from external repositories, and any control that gates compliance decisions should require explicit security review as part of the merge process.

Defense in depth

- Any control that gates a compliance or security decision should be implemented as fail-closed by default.
- Content Security Policy should be tightened across the application. A tightly-scoped CSP is a cheap control that substantially reduces the blast radius of any future injection issue.
- A bug bounty or responsible-disclosure program, if one is not already in place, provides continuous external coverage between annual penetration tests and gives researchers who would otherwise have no authorised channel a safe way to report issues. Even a lightweight responsible-disclosure policy, a public security.txt, a monitored security contact, and a clear statement of safe-harbour intent captures most of the value at minimal operational cost and is a strong signal to the broader security community that reports are welcome. A paid bounty program can be layered on top later once the intake and triage process is established.

Vulnerability details



High severity findings

1.1 Geo Location blocking bypass via Spoofed x-country-code Header

Finding details

All API endpoints use a client-supplied x-country-code request header to enforce sanctions/compliance country blocking. This header is never validated as originating from Cloudflare and can be freely modified or omitted by any client. By setting x-country-code to a non-sanctioned value, or by omitting the header entirely, a user in an embargoed jurisdiction can obtain access to endpoints that should be blocked.



```
(kali㉿kali)-[~/Desktop/euler]
$ curl -s -o /dev/null -w "%{http_code}" -X POST https://euler-lite.euler.finance/api/screen-address \
-H "Content-Type: application/json" \
-H "x-country-code: IR" \
-H "X-Forwarded-For: 10.0.0.1" \
-d '{"address": "0x1234567890123456789012345678901234567890"}'
451

(kali㉿kali)-[~/Desktop/euler]
$ curl -s -o /dev/null -w "%{http_code}" -X POST https://euler-lite.euler.finance/api/screen-address \
-H "Content-Type: application/json" \
-H "X-Forwarded-For: 10.0.0.1" \
-d '{"address": "0x1234567890123456789012345678901234567890"}'
200

(kali㉿kali)-[~/Desktop/euler]
$ curl -s -o /dev/null -w "%{http_code}" -X POST https://euler-lite.euler.finance/api/screen-address \
-H "Content-Type: application/json" \
-H "x-country-code: US" \
-H "X-Forwarded-For: 10.0.0.1" \
-d '{"address": "0x1234567890123456789012345678901234567890"}'
200
```

Figure 1: Demonstration of how changing/omitting the x-country-code affects the requests

Security Impact

This is a sanctions/compliance issue first and a security issue second. Because the control can be bypassed with a trivial header change, the organization cannot rely on it as evidence of geographic access enforcement. Depending on jurisdiction, allowing access to users in sanctioned countries can result in regulatory action, fines, and reputational harm.

Remediation

- Stop trusting the x-country-code header. Read [Cloudflare's CF-IPCountry header](#), which is set by the Cloudflare edge and cannot be spoofed by a client reaching the origin through Cloudflare.
- Strip all client-supplied x-country-code and other geo-origin headers at the ingress layer before they reach application code.

VectorSec Penetration Testing Report

- Switch to a **fail-closed model**: when country cannot be determined (missing or malformed CF-IPCountry), deny access rather than allow it.
- If non-Cloudflare environments must be supported, resolve country from the validated client IP using a server-side GeoIP lookup, never from a client-controlled header.

1.2 Incomplete and Fail-Open OFAC Address Screening Coverage

Finding details

The address-screening API is intended to flag sanctioned addresses, including Tornado Cash-related addresses, before users can interact with the application. Testing identified both a coverage gap and a fail-open behavior:

- Not all sanctioned Tornado Cash addresses are currently flagged as suspicious by the upstream TRM Labs provider via this integration.
- If WALLET_SCREENING_URI is not set, screening silently passes.
- If the TRM Labs API returns a non-2xx HTTP response, screening passes.
- If the upstream request hits a network error or the 5-second timeout, screening passes.

```
1 POST /api/screen-address HTTP/2
2 Host: euler-lite.euler.finance
3 Cookie: cf_clearance=
  XRBfwbXAOgD0eBVA8RtuW3LwPHK4ARXYOAXqZGh6qU-1774985684-1.2.1.1-iMniA4K9
  j90vODccdzDejwWzqBI6VDvanlQvwSW9xLDqI VYo8uVdmm4_TZGTvwvyX00WdyKRZOAG6ttO
  DgJvFjBd2yWw_Sr.i4dWerwY7Jev82dtlcyg.qV7x2lgJU540zI5foKwGzZDMPQwOPq2Ng
  mV6MvoedL_7oKJZ6vsziavEFIPW89rk.xtap4NaWaAtSesB.QYvyCYDNBLAMN_2sUwWwy4Z
  gedECqFd2L71.w; __cf_bm=
  uzN8EHRTXt519jUFmqTvsCaSBCub9.or_7gygaYC_8o-1774985684.9790008-1.0.1.1-
  DKEBHmjGzLcQANXTMmtjcI3wvcjwY0ig83SLUyEM3dykaIvJh6QbtLzAZtHaXmLDBWdtT
  seERb4y6yJ6ZG4ve.GhmQAVJhaVLXceIuEg.Zf1ViAB6ctNjsSSSL_1Bw
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
  Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Referer: https://euler-lite.euler.finance/explore?network=1
9 Content-Type: application/json
10 Content-Length: 74
11 Origin: https://euler-lite.euler.finance
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=4
16 Te: trailers
17
18 {
  "address": "0x8589427373D6D84E98730D7795D8f6f8731FDA16",
  "vpnIsUsed": false
}
```

```
1 HTTP/2 200 OK
2 Date: Tue, 31 Mar 2026 21:09:02 GMT
3 Content-Type: application/json
4 Content-Length: 29
5 Access-Control-Allow-Headers: Content-Type
6 Access-Control-Allow-Methods: POST, OPTIONS
7 Access-Control-Allow-Origin: https://euler-lite.euler.finance
8 X-Railway-Cdn-Edge: fastly/cache-sof1510048-SOF
9 Permissions-Policy: geolocation=(), microphone=(), camera=()
10 Referrer-Policy: same-origin
11 Server: cloudflare
12 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
13 Vary: Origin
14 X-Content-Type-Options: nosniff
15 X-Frame-Options: SAMEORIGIN
16 X-Railway-Edge: railway/europe-west4-drams3a
17 X-Railway-Request-Id: LHrYg7i3S7S-FiQYvEOTxA
18 X-Cache: MISS
19 X-Cache-Hits: 0
20 X-Served-By: cache-sof1510048-SOF
21 Cf-Cache-Status: DYNAMIC
22 X-Xss-Protection: 1; mode=block
23 Set-Cookie: __cf_bm=
  CQviR.BU3XfUzNI1Q.u8YESLhJKVst5uKuQhDrGcLVU-1774991342.1064792-1.0.1.1-
  CzBQOku0mndKuu3i5LkWiKGrTckhgCenPRUzgcjQ9DTR.ucooVck0RUSUqZ65d19bGhoqu
  EJ6bQckB_YbjpoOX7kpGqiYyYH.r_jqAgxp3ax4lqZRNsjfBOBSLPnviV; HttpOnly;
  Secure; Path=/; Domain=euler.finance; Expires=Tue, 31 Mar 2026 21:39:02
  GMT
24 Expect-Ct: max-age=86400, enforce
25 Cf-Ray: 9e5255302e4955ad-BEG
26
27 {
  "addressIsSuspicious": false
}
```

Figure 2: Tornado Cash sanctioned address reported as non-suspicious

Security Impact

Any of these conditions allows a sanctioned address to interact with the application without being blocked. The consequences are primarily regulatory, the organization has a screening control in place that, under a wide range of realistic failure modes, does not deny access. An attacker or a sanctioned user does not need to 'exploit' anything in the classical sense, they only need the upstream service to be briefly misconfigured, unreachable, or slow.

Remediation

- Engage with TRM Labs to understand why specific sanctioned Tornado Cash addresses are not returned as suspicious and either update the integration (endpoint, list, or query) or add a supplementary OFAC SDN list check.
- Switch the screening code path to fail-closed: treat missing configuration, non-2xx responses, network errors, and timeouts as 'suspicious' rather than 'cleared'.
- Add health checks and alerting on the screening provider so that any degradation of the upstream service is visible to operations rather than silently bypassing the control.
- Log screening decisions (success, failure, fail-closed) with enough metadata to support compliance audits.

1.3 Exposed RPC API Keys in Frontend JavaScript

Finding details

RPC provider API keys, including keys for premium archive nodes, are present in the frontend JavaScript bundle delivered to every visitor. Anyone viewing the site can extract these keys from the browser devtools or from a saved copy of the bundle and reuse them from any client.

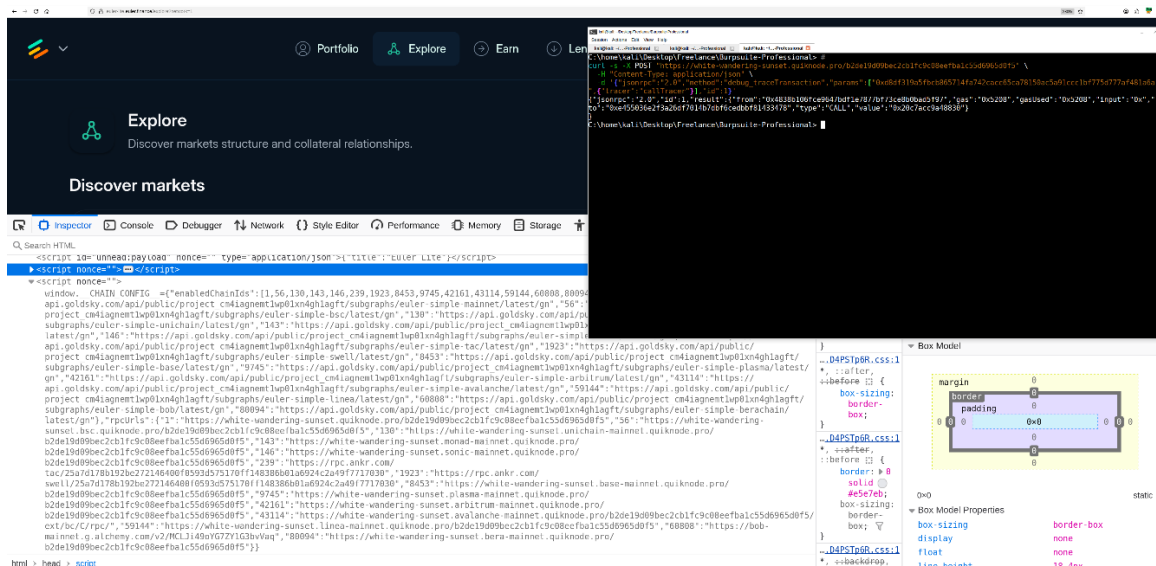


Figure 3: RPC Keys showing in frontend code

Security Impact

Archive-node access is a premium tier with significantly higher per-request pricing than standard RPC. With the keys exposed, any third party can use them to issue arbitrary RPC calls at the organization's expense. Depending on the provider's billing model this can result in direct financial loss, quota exhaustion that degrades legitimate user experience, or provider account suspension.

Remediation

- Remove all RPC API keys from frontend JavaScript. Proxy RPC calls through a server-side endpoint that holds the keys in a [secret store](#) and applies authentication, rate limiting, and method allow-listing.
- Rotate every key that has been shipped in the frontend bundle. Consider any historical key exposed on the internet as burned.
- Add monitoring for unusual usage patterns on RPC providers (traffic from unexpected IP ranges, sudden spend increases) so that future exposure is detected quickly.
- For any keys that genuinely must be embedded client-side, use provider features that restrict usage by referrer/origin and per-key quotas.

1.4 Unauthenticated `eth_sendRawTransaction` Relay via JSON-RPC Proxy

Finding details

The JSON-RPC proxy enforces a method allow-list but applies no caller authentication, no CSRF token, and no Origin check at the handler layer. The allow-list includes `eth_sendRawTransaction`. Requests from arbitrary origins are accepted and forwarded to the upstream Tenderly node, while unsupported methods are correctly rejected. Any website, script, or non-browser client that can reach the server can use the proxy as a free transaction relay to broadcast arbitrary attacker-signed transactions on-chain.

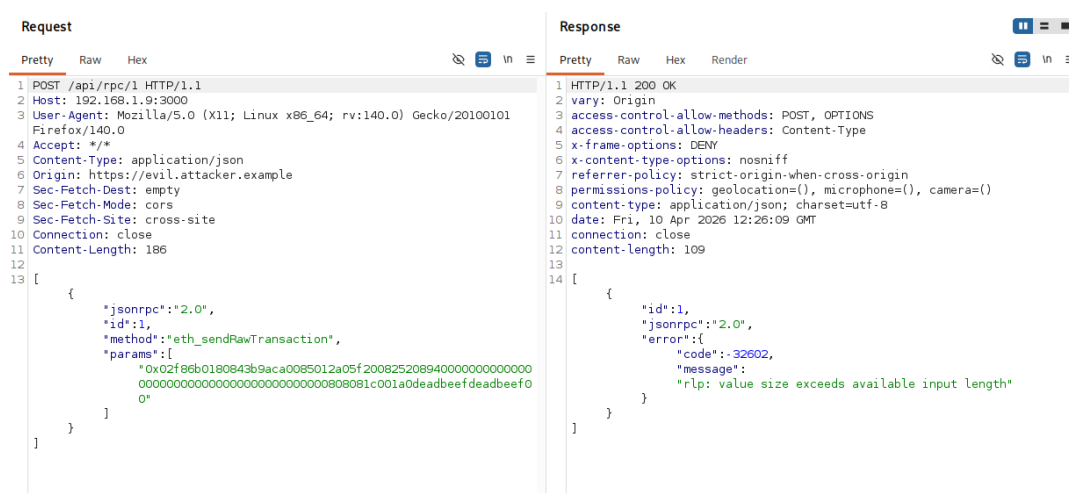


Figure 4: Normal request showing that request arrives to the Tenderly node

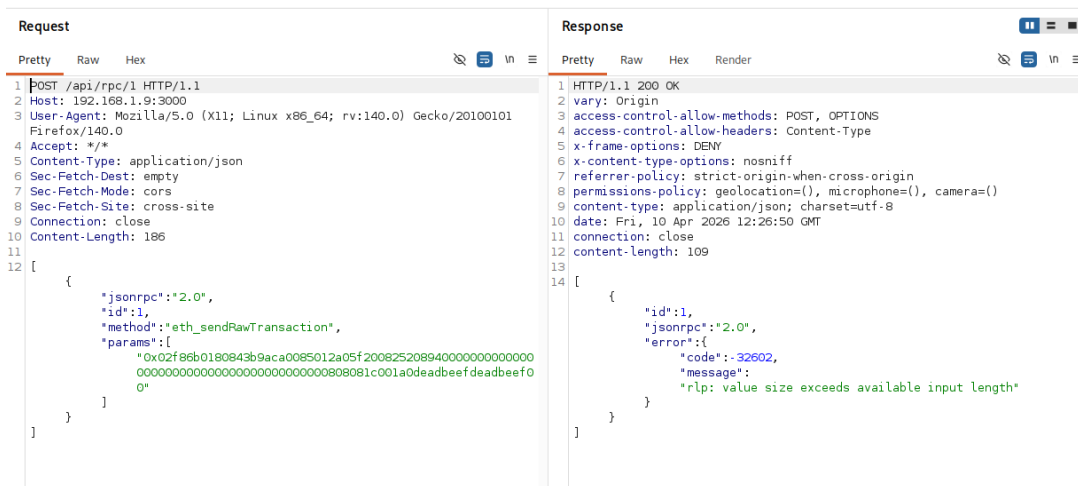


Figure 5: Request with malicious origin arrives to the Tenderly node as well

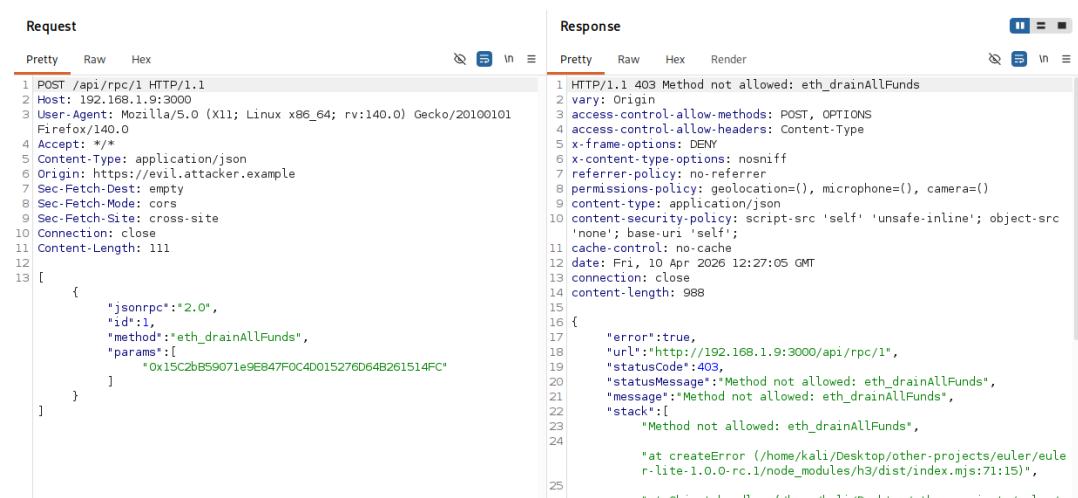


Figure 6: Unsupported method request does not work

Security Impact

End-user funds are not directly at risk through this vector, because the attacker must sign their own transactions. However, the issue has two concrete impacts on the organization. First, the proxy and its upstream RPC provider absorb the cost of every broadcast, which creates an unauthenticated financial loss channel via quota abuse. Second, broadcasts issued through the proxy appear to originate from the company server's IP address rather than the attacker's. This provides IP-origin laundering that can be used to evade provider bans, bypass IP blocklists, or obscure attribution during MEV attacks and sybil activity.

Remediation

- Highest-leverage single change: remove `eth_sendRawTransaction` from the RPC method allow-list. The proxy should not broadcast transactions on behalf of arbitrary callers.
- Enforce Origin checks at the handler layer (not only in middleware) by reading `getRequestHeader(event, 'origin')` at the top of the RPC handler and rejecting requests whose origin is not in the allow-list. Reject state-changing requests that omit the Origin header entirely, since same-origin browser POSTs always include it.
- Partition rate limits per-method. Read methods like `eth_call` legitimately need thousands of requests per minute, but any write methods that remain need only a handful; per-method limits prevent write-heavy abuse from hiding inside read-method budgets.
- If broadcasting must be supported for a specific use case, gate it behind an authenticated endpoint tied to an application session, not the generic RPC proxy.

1.5 Stored XSS Chain via Compromised Labels Repository (6 sinks)

Finding details

Six stored XSS vulnerabilities were identified in the frontend rendering of label data consumed from the `euler-xyz/euler-labels` GitHub repository. All six issues share two root causes and manifest as distinct injection sinks across multiple pages. **Exploitation requires write access to the labels repository**, either through stolen developer credentials, a malicious pull request merged without adequate review, or a CI/CD supply-chain attack.

Root cause 1 – URL encoding gap in `autoLink()`

The `autoLink()` helper in `utils/autoLink.ts` processes markdown-style links of the form `[text](url)` and produces HTML. The link text is correctly passed through `escapeHtml()`, but the captured URL (`match[2]`) is inserted raw into the `href` attribute:

```
const LINK_RE = /\[[^\]]+\]\(\s*((https?:\/\/[^\s]+)\s*)\)/g
// ...
formatted += `<a href="${match[2]}" target="_blank" ...>${escapeHtml(match[1])}</a>`
```

The URL character class `[^\s]+` excludes whitespace and closing parenthesis but still permits a double-quote character. An attacker can inject a `"` inside the URL to close the `href` attribute early and append arbitrary additional HTML attributes, most notably a `style` attribute that is then rendered by the browser.

Root cause 2 – Unvalidated URL in `:href` binding

The `normalizeEntities()` function in `utils/eulerLabelsUtils.ts` copies `entity.url` verbatim using object spread (`...entity`) without any protocol validation, and the resulting value is bound directly to `:href` in Vue templates. The application has a correct reference implementation in `VaultPointsModal.vue` that uses an `isValidUrl()` helper with an explicit `http://https:` allow-list, but this check is not applied to

entity URLs. As a result, attacker-supplied javascript: and data: URIs are rendered verbatim in the DOM.

Injection sinks

- VULN-01.a – earn-vaults.json deprecationReason: fires on deprecated vault pages.
- VULN-01.b – earn-vaults.json description: fires on any earn vault overview page, no wallet required.
- VULN-01.c – products.json description (three component sinks): fires on vault detail pages across multiple components, including per-vault overrides via vaultOverrides[vaultAddr].description.
- VULN-01.d – products.json deprecationReason: fires on deprecated vault pages when the vault is listed in deprecatedVaults[].
- VULN-01.e – portfolioNotice: the most severe sink; renders on the portfolio page for every user with an active position in the targeted vault, enabling a full-page phishing overlay matching the application's dark theme.
- VULN-01.f – entity.url javascript:/data: URI injection via Vue :href binding.

A proof of concept was developed for the description sink (earn-vaults.json description) and tested in a local environment. XSS-PoC script will be sent as an attachment to the report.

```
(root@kali) ~/Desktop/other-projects/euler/xss
node exploit.js
[1] Bypassing onboarding ...
[2] Finding a vault address from /earn ...
[*] Screenshot saved: 01-earn-listing.png
[+] Target vault: 0xb072b2779f1e1a6a9d2d5faa1766f341b92a83a
[3] Setting up labels API intercept ...
[4] Navigating to http://127.0.0.1:3000/earn/0xb072b2779f1e1a6a9d2d5faa1766f341b92a83a
[intercept] Injecting XSS payload into earn-vaults.json
[*] Screenshot saved: 02-vault-before-check.png
[5] Checking DOM for injected overlay ...

[+] === EXPLOITED - Full-page overlay confirmed! ===
{
  "tag": "A",
  "class": "",
  "style": "position: fixed; top: 0; left: 0; width: 100vw; height: 100vh; background: #0d0d1a; color: #ffffff; font-size: 28px; display: flex; align-items: center; justify-content: center; z-index: 2147483647; text-decoration: none; cursor: pointer;",
  "text": "SECURITY-ALERT-VERIFY-WALLET"
},
{
  "tag": "DIV",
  "class": "",
  "style": "position: fixed; inset: 0px; z-index: 999999; pointer-events: none; opacity: 0; transition: 0.15s;",
  "text": ""
},
{
  "tag": "svg",
  "class": {},
  "style": "position: fixed; inset: 0px; pointer-events: none;",
  "text": ""
},
{
  "tag": "DIV",
  "class": "",
  "style": "position: fixed; top: 0px; left: 0px; transition: 0.15s; background-color: rgb(17, 153, 119); color: rgb(255, 255, 255); border-bottom-left-radius: 4px; border-bottom-right-radius: 4px; padding: 1px 4px; pointer-events: none; font-size: 10px; font-family: monospace; display: flex; gap: 0.25rem; box-shadow: rgba(0, 0, 0, 0.2) 0px 0px 2px; opacity: 0;",
  "text": ""
},
{
  "tag": "NUXT-DEVTOOLS-INSPECT-PANEL",
  "class": "",
  "style": "z-index: 999999; position: fixed;",
  "text": ""
}
}
[*] Proof screenshot saved: 03-EXPLOITED.png
[*] Done. Check screenshots in /home/kali/Desktop/other-projects/euler/xss
```

Figure 7: Running of the custom script made for exploitation

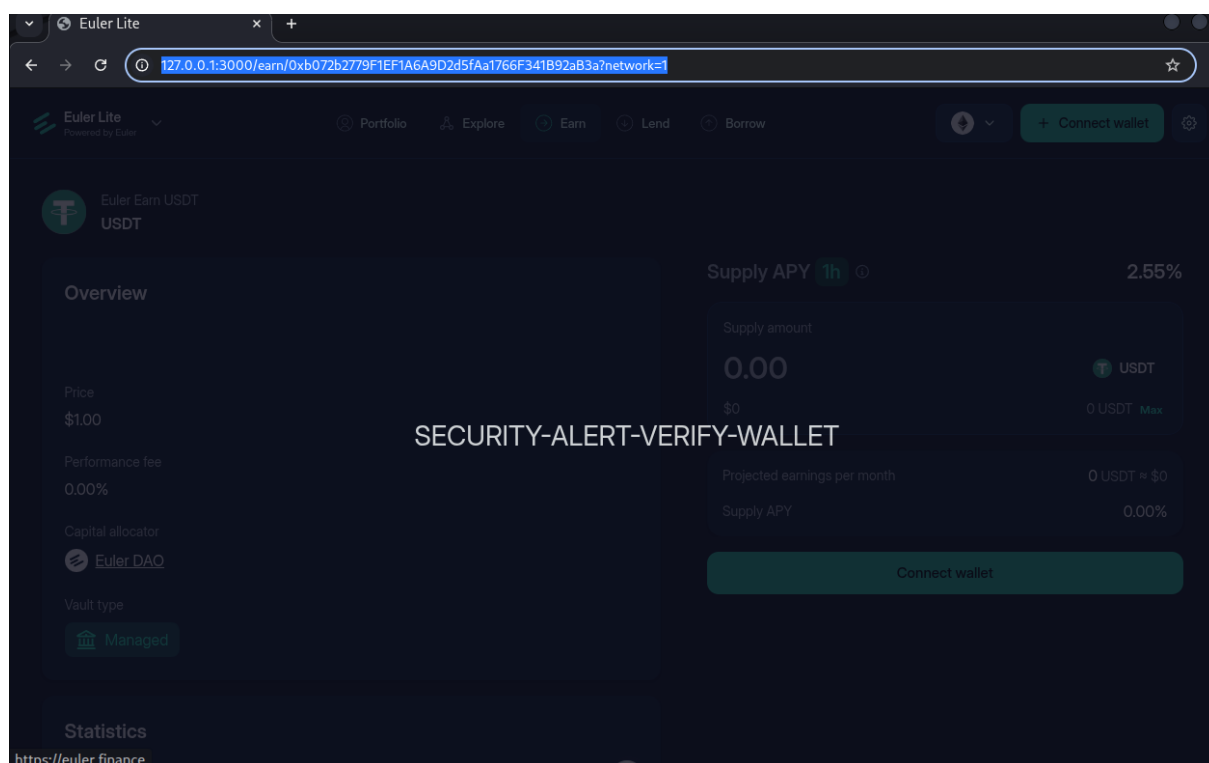


Figure 8: XSS exploited in the local environment

Security Impact

Exploitation delivers a full-page phishing surface on users' portfolio pages, the single highest-value page for active DeFi users. Attackers can prompt for wallet signatures, malicious transaction approvals, or seed phrases, with the phishing UI served from the legitimate origin. The sinks on public pages (description fields) do not require a connected wallet and can be used for broad reconnaissance or social-engineering prior to a targeted attack. While Vue 3.5.30 is generally expected to sanitize javascript: URIs, testing confirmed that it does not, and the application has no defense in depth against this.

Remediation

- In `utils/autoLink.ts`, apply `escapeHtml()` to `match[2]` before it is inserted into the `href` attribute. As a defense-in-depth measure, also add a double-quote to the regex exclusion class so the class becomes `["^\"s"]+`.
- In `utils/companyLabelsUtils.ts` `normalizeEntities()`, validate URLs with an explicit `http://https:` allow-list matching the `isValidUrl()` helper already used in `VaultPointsModal.vue`.
- Server-side, add JSON-schema validation in `server/api/labels/[file].get.ts` to reject any label-file strings that contain a double-quote, `javascript:`, or `data:` in URL-context fields before the proxy caches the response.
- Tighten Content-Security-Policy by replacing `style-src 'unsafe-inline'` with a [nonce-based](#) `style-src 'nonce-...'`. This removes the residual capability for injected style attributes to render even if a similar injection bug recurs.

VectorSec Penetration Testing Report

- Treat the labels repository as a production artifact: enforce branch protection, required code review, signed commits, and CI provenance so that a supply-chain compromise of the repository is itself harder to achieve.

Medium severity findings

1.6 API Rate-Limit Bypass via X-Forwarded-For Manipulation

Finding details

Application-layer rate limits are defined for /api/ endpoints, with the strictest being 10 requests per 60 seconds on /tenderly/simulate and /screen-address. Cloudflare enforces an additional request-rate limit at the edge. The per-IP rate limit in the application uses the X-Forwarded-For header to identify the caller, and this header is trusted without validation. An unauthenticated attacker can rotate the X-Forwarded-For value every few requests to reset the per-IP counter. Cloudflare's rate limit can be bypassed by slowing the request cadence or by reaching the origin directly via its IP address and bypassing Cloudflare entirely.

In a local environment without Cloudflare, a script demonstrated that 30 consecutive requests to a rate-limited endpoint were not blocked. In production, Cloudflare currently terminates abusive traffic with HTTP 429, which limits the practical impact today. The underlying trust issue in the application, however, remains and is relevant to any environment where Cloudflare is not in front of the origin.

```
(kali@kali)-[~/Downloads]
$ for i in $(seq 1 12); do
  STATUS=$(curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:3001/api/screen-address \
    -H "Content-Type: application/json" \
    -H "X-Forwarded-For: 1.1.1.100" \
    -d '{"address": "0x1234567890123456789012345678901234567890"}')
  echo "Request $i: HTTP $STATUS"
done
Request 1: HTTP 200
Request 2: HTTP 200
Request 3: HTTP 200
Request 4: HTTP 200
Request 5: HTTP 200
Request 6: HTTP 200
Request 7: HTTP 200
Request 8: HTTP 200
Request 9: HTTP 200
Request 10: HTTP 200
Request 11: HTTP 429
Request 12: HTTP 429

(kali@kali)-[~/Downloads]
$ curl -s -X POST http://localhost:3001/api/screen-address \
  -H "Content-Type: application/json" \
  -H "X-Forwarded-For: 2.2.2.200" \
  -d '{"address": "0x1234567890123456789012345678901234567890"}'
# Result: HTTP 200 - {"addressIsSuspicious": false} - RATE LIMIT BYPASSED
{
  "addressIsSuspicious": false
}
```

Figure 9: Request showing that changing the X-Forwarded-For header can bypass the rate-limit

Security Impact

In a production environment protected by Cloudflare, the attacker must additionally throttle requests or bypass Cloudflare to exploit the bypass, which lowers the practical severity. In any environment where the origin is reachable without Cloudflare (non-Cloudflare deployments, origin IP exposure, staging environments), an unauthenticated attacker can exhaust upstream quotas on paid services such as Tenderly simulations and screening providers, driving up operational cost.

Remediation

- If non-Cloudflare environments are in scope, strip X-Forwarded-For at the ingress boundary and let the application derive the client IP from the transport layer, then use that value for rate limiting.
- If only Cloudflare-fronted environments are supported, stop consuming X-Forwarded-For entirely and rate-limit on CF-Connecting-IP, which Cloudflare sets at its edge and which a client reaching the origin through Cloudflare **cannot spoof**.
- Harden the origin so that it is not reachable directly by its public IP (restrict inbound to Cloudflare IP ranges, or use an authenticated origin pull).

1.7 Unauthenticated Access to /api/tenderly/simulate

Finding details

The `/api/tenderly/simulate` endpoint accepts requests without any caller authentication. The endpoint proxies simulation requests to Tenderly, a paid upstream service billed on a per-simulation basis. Any unauthenticated caller can invoke simulations, including automated scripts, at the organization's expense.

[illegible]

Figure 10: API request to the Tenderly simulate endpoint

VectorSec Penetration Testing Report

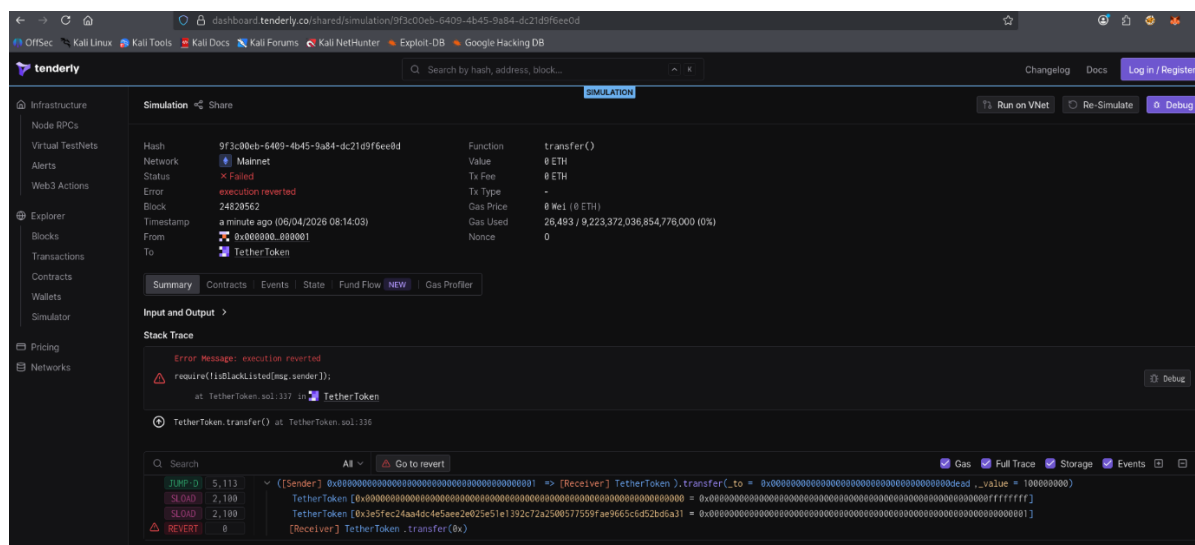


Figure 11: Tenderly url showing transaction details

Security Impact

This is primarily a cost-abuse issue. A malicious actor, or simply an automated scraper, can consume the Tenderly simulation budget and cause direct financial loss. Combined with the Rate-limit bypass vulnerability, the cost ceiling can be raised substantially. The simulation endpoint also exposes detail about the application's simulation workflow to external callers, though the sensitivity of that surface is limited.

Remediation

- Require an authenticated application session to call `/api/tenderly/simulate`, or at minimum an Origin check that rejects requests not coming from the application's own web UI.
- Apply per-session or per-account quotas on top of per-IP rate limits so that a single session cannot exhaust the simulation budget.
- Add server-side budgeting and alerting on Tenderly spend so that abnormal simulation volume is detected before it becomes costly.
- Cache simulation results for identical inputs where semantics permit, reducing the marginal cost of repeat requests.

Low severity findings

1.8 Client-Side OFAC Screening Cache Race Condition

Finding details

The frontend composable `composables/useAddressScreen.ts` performs OFAC screening on the connected wallet address by calling the `/api/screen-address` backend. To avoid duplicate calls when the same wallet stays connected across re-renders or chain switches, the composable caches the most recently screened address in a module-level plain let variable (`lastScreenedAddress`). On subsequent screening invocations, if the new address matches the cached value, the function short-circuits and returns `false` (not restricted) without calling the backend. The cache is cleared only by a separate `Vue watch(wagmiAddress, ...)` watcher that invokes `resetScreeningCache()` when `wagmiAddress` transitions to undefined.

Two code paths can trigger screening for the same address in close succession: the `wagmi useAccountEffect({ onConnect })` callback and the address watcher at `useWagmi.ts:56-63`. Because `lastScreenedAddress` is a plain let and not a reactive primitive, its reads and writes are not synchronized with Vue's reactivity flush. The disconnect button calls `disconnect()` without awaiting it and immediately closes the modal, allowing the user to initiate a reconnection before the disconnect's reactive propagation has fully run. In a worst-case interleaving, for example, if a queued microtask causes the reconnection's `onConnect` to fire before the watcher's `resetScreeningCache()` runs, `screenConnectedAddress("0xSanc")` sees `lastScreenedAddress === "0xsanc"` from the previous session, returns `false` immediately, and the wallet is considered cleared without ever contacting the screening API.

Security Impact

In the worst case, a sanctioned wallet that was previously screened in the same page session could reconnect after a disconnect and skip OFAC re-validation entirely, allowing it to interact with the lending UI as a cleared address. In practice this requires a very specific reactive scheduling window and assumes the backend screening would otherwise have blocked the address. The finding is also strictly client-side, a correctly implemented server-side enforcement layer must independently re-validate the address on every state-changing request, which reduces the issue to a defense-in-depth concern rather than a direct control bypass. The severity is therefore Low, but the issue is worth fixing to eliminate the class of stale-cache bugs and to keep client-side behavior aligned with server-side policy.

Remediation

- Minimal fix: in `useAddressScreen.ts` convert `lastScreenedAddress` from a module-level let into a [reactive ref](#), and tie its lifetime to the connected address rather than to a separate watcher.

VectorSec Penetration Testing Report

- Structural fix: remove the deduplicate entirely and re-screen on every connect. The cost is one HTTP round-trip, which is negligible compared to the RPC calls already issued per page load, and it eliminates the entire class of stale-cache bypass bugs.
- If the deduplication must be kept for performance, scope it per-wagmi-session by clearing `lastScreenedAddress` in a `watchEffect` that depends on `wagmiAddress` rather than in a separate watcher, so the reset and the next screening invocation are guaranteed to fire in the same reactive cycle.
- Ensure that the server-side `/api/screen-address` enforcement is the source of truth, and that no sensitive, state-changing action on the backend relies on the frontend's screening cache.